



SCHAEFFLER

Harmonic-Drive Digital Twin Package Anatomy

What is inside the release · piece by piece · in plain language

Author:	Lu Zhu
Date:	2026
Status:	Rev 1.0 — Internal

We pioneer motion

Contents

1	Read this first	2
2	The map	2
3	The four layers	3
3.1	Layer 1 — flex_common/ : the physics foundation	3
3.2	Layer 2 — params.py : the dials	4
3.3	Layer 3 — twin.py : the engine	4
3.4	Layer 4 — __init__.py : the front door	5
4	The reality check	5
4.1	catalogue.py (250 lines)	5
4.2	tests/test_catalogue_validation.py (183 lines)	6
5	The on-ramps — examples/	6
6	How a user actually meets the package	7
7	Before you release — three things to look at	7
8	One-page summary	8

1 Read this first

This document explains **what you are shipping** when you release harmonic-drive-twin, piece by piece, in plain language.

It is written for two readers: **you**, deciding what belongs in the release; and **a stranger** who has just cloned the repository and wants to know what they are looking at.

The whole package in one paragraph

The package is a **physics calculator for a strain-wave gearbox**. You hand it an operating point — “I need 30 N m out, the motor is spinning at 2000 rpm, it is 20 °C in there” — and it hands back the motor torque you actually need, the efficiency, where every watt of loss went, and how much the gearbox twists under that load. Everything else in the repository exists to support, prove, or demonstrate that one calculation.

Its companion, the *User Guide*, tells a reader **how to drive**. This document tells them **what is under the hood**.

2 The map

```

harmonic-drive-twin/
|
├─ pyproject.toml      ← the shipping label (name, version, deps)
├─ LICENSE             ← MIT – permission to use it
├─ README.md          ← the shop window (what it does, quick start)
├─ .gitignore          ← what never leaves your machine
|
├─ harmonic_drive_twin/ ← THE PRODUCT – all of the model is here
|   │
|   │   └─ __init__.py      the front door
|   │   └─ twin.py         the engine
|   │   └─ params.py       the dials
|   │   └─ catalogue.py    the reality check
|   │   └─ py.typed        flag saying "type hints included"
|   │   └─ flex_common/    the physics foundation (7 modules)
|   │       │
|   │       │   └─ baseline.py  ellipse.py  mesh.py  ring.py
|   │       │   └─ friction.py  profile.py  teeth.py
|   │
|   └─ examples/          ← on-ramps: three runnable scripts
|       │
|       │   └─ basic_usage.py
|       │   └─ isaacsim6_actuator.py
|       │   └─ isaacsim6_pendulum.py
|
|   └─ tests/              ← the proof it still works
|       │
|       │   └─ test_catalogue_validation.py
|
|   └─ docs/               ← the manual
|       │
|       │   └─ user_guide/
|       │       │
|       │       │   └─ user_guide.pdf      10-page branded guide
|       │       │   └─ package_anatomy.pdf this document
|       │       │   └─ assets/             logos + pendulum result figure

```

Total: **2 947 lines of Python** in the model itself (3 633 counting examples and tests). Small — and that is a feature. A reviewer can read the whole model in an afternoon.

3 The four layers

The package is built in layers. Each layer knows only about the ones below it. This is why it stays understandable.

4		<code>__init__.py</code>		the front door		← users import this
3		<code>twin.py</code>		the engine		← does the work
2		<code>params.py</code>		the dials		← what you may change
1		<code>flex_common/</code>		the physics foundation		← the equations

`catalogue.py` sits alongside 2–3 and asks: "does this match reality?"

3.1 Layer 1 — `flex_common/` : the physics foundation

Roughly 1 300 lines across seven small modules. Each is one piece of textbook mechanics.

These are the equations describing *how a harmonic drive deforms and meshes*. They began as the shared library of the multi-topic FEA study and were bundled into the package so it stands alone — a user does **not** need the research repository to run the model.

Module	Lines	In one sentence
<code>baseline.py</code>	79	The reference part. One agreed set of numbers — $R = 20$ mm, wall 0.4 mm, ovalization 0.30 mm, 200/202 teeth — so every other module is talking about the same gearbox.
<code>ring.py</code>	170	The flexspline as a thin ring. Squash a thin steel ring into an ellipse; this gives the bending moment, shear and contact pressure in closed form. Handles both “tied to the cam” and “cam can only push outward”.
<code>ellipse.py</code>	111	A real ellipse is not a pure $\cos 2\theta$. Decomposes the true elliptical shape into harmonics and adds the ring response to each. The correction term for <code>ring.py</code> .
<code>mesh.py</code>	305	Which teeth are touching, right now. Given the wave-generator angle it says which flexspline teeth line up with which circular-spline spaces, and how each tooth moves: radial breathing, tangential slide, tilt. The heart of the kinematics.
<code>profile.py</code>	433	What shape the teeth must be. Synthesises the circular-spline flank as the conjugate envelope of the flexspline tooth swept through the mesh, and computes the flank sliding speed — which feeds friction and wear.
<code>friction.py</code>	74	Why it takes torque to spin an unloaded drive. Without friction the no-load torque is exactly zero; this shows why, and what friction adds.
<code>teeth.py</code>	184	Tooth-root stress. Lewis cantilever bending plus Dolan–Broghamer fillet concentration — the fatigue surrogate.

Gentle note: a user of the twin never needs to open these. They are the foundation under the floorboards. But they are what makes the model *physics-based* rather than a curve fit — and a skeptical reviewer will go straight here.

3.2 Layer 2 — params.py : the dials

425 lines. Four dataclasses. This is “everything you are allowed to change”.

Instead of magic numbers scattered through the code, every adjustable quantity lives in one of four labelled boxes.

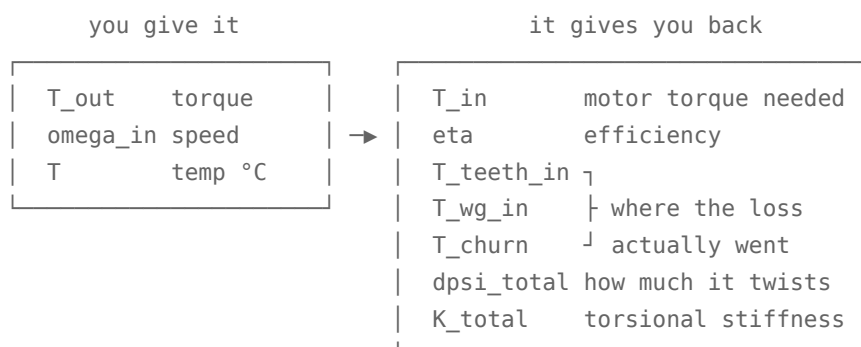
Dataclass	What it holds	Example knobs
FrictionParams	How things rub	boundary friction coefficient, Stribeck transition speed, wave-generator bearing coefficient, grease churn factor
ThermalParams	How temperature changes things	oil viscosity versus temperature, thermal expansion of clearances
ToleranceParams	Manufacturing imperfection	backlash, tooth-to-tooth pitch scatter (used for Monte-Carlo runs)
StiffnessChain	How the drive twists	the series chain: cup + wave-generator bearing + mesh + output bearing

Why this matters for release: a user adapting the twin to *their* gearbox edits parameters here — never the equations. That is the contract. Units are stated once, at the top of the file, and hold everywhere: **mm, N, N mm, rad/s, °C**.

3.3 Layer 3 — twin.py : the engine

787 lines. One class, *HarmonicDriveTwin*, plus three contact helpers.

This is the actual model. It takes the equations from Layer 1, the numbers from Layer 2, and answers questions. The main method is `solve()`:



The loss split into **three channels** is the model’s central idea:

1. `T_teeth_in` — tooth flanks rubbing. Boundary-lubricated, not oil-film.
2. `T_wg_in` — the wave-generator bearing rolling.

3. T_{churn} — grease churn and seal drag. Load-independent, and at rated torque it is the **biggest** of the three.

Why the third channel matters

The load-free churn/seal channel is the non-obvious finding the study produced, and it is why catalogue efficiency sits at 67–78 % rather than the $\approx 90\%$ a tooth-friction-only model predicts.

Beyond `solve()`, the class also answers:

Method	Question it answers
<code>efficiency_curve()</code>	How does η vary with torque?
<code>efficiency_map()</code>	2-D η map over torque $\times$ speed
<code>hysteresis()</code> / <code>hysteresis_dahl()</code>	What does the torque–windup loop look like? (lost motion)
<code>breakaway_input_torque()</code>	What torque starts it from cold rest?
<code>backdrive_threshold()</code>	Can the load push the motor backwards?
<code>monte_carlo()</code>	What is the scatter if tooth clearances vary?
<code>warmup()</code>	Thermal transient and grease life
<code>wear_life()</code>	Archard flank wear over a lifetime
<code>hertz_flank_summary()</code>	Contact pressure and oil film at the hardest-loaded tooth

Plus three standalone helpers anyone can use: `hertz_flank()`, `ehl_lambda()` and `wg_bearing_stiffness()`.

3.4 Layer 4 — `__init__.py`: the front door

129 lines, almost all of it documentation. Its job is curation:

1. **Decides what is public.** The `__all__` list is the promise: these 13 names are the supported API. Everything else is internal and may change.
2. **Provides `solve_point()`** — a one-line functional wrapper so a casual user never has to instantiate a class: `solve_point(T_out_Nm=30.0, speed_rpm=2000)  $\rightarrow \eta = 77.9\%$` .
3. **Carries the quick-start docstring**, written as doctests — so the examples in the documentation are *executable* and cannot silently go stale.

4 The reality check

This is what separates a digital twin from a plausible-looking spreadsheet.

4.1 `catalogue.py` (250 lines)

Holds the digitised **Schaeffler TPI 275** table (series RT1-H-CS, $i = 100$, $+20^\circ\text{C}$) for four frame sizes:

Symbol	Quantity	14	17	20	25
R	pitch radius [mm]	17.78	21.59	25.40	31.75
T_N	rated torque [N m]	10.0	31.0	52.0	87.0
η	efficiency [%]	67.0	77.0	77.0	77.0
T_{nlrt}	no-load running torque [mN m]	35	51	105	195
T_{nlst}	starting torque [mN m]	21	29	37	69
T_{bt}	back-driving torque [N m]	2.21	3.06	3.89	7.26
K_1	torsional stiffness, low [N m/rad]	4 700	10 000	16 000	31 000
K_3	torsional stiffness, high [N m/rad]	7 100	16 000	29 000	57 000

And three functions: `twin_for_size(14|17|20|25)` returns a twin pre-scaled to that catalogue size; `calibrate()` derives the three global calibration dials from the table; `validate()` runs everything and reports how close the model lands.

The headline claim of the whole package

Five independent catalogue quantities, across four sizes, matched with only **three global calibration dials** and **zero per-size fitting**. Torsional stiffness uses no fitting at all — it comes from the FE-anchored physical chain.

4.2 tests/test_catalogue_validation.py (183 lines)

Four tests that keep that claim honest:

Test	Guards against
<code>test_calibration_reproducible()</code>	calibration drifting when code changes
<code>test_solve_point_size17()</code>	the size-17 number (75.8% versus catalogue $77 \pm 3\%$) moving
<code>test_solve_point_default()</code>	the baseline number (77.9%) moving
<code>test_twin_for_invalid_size()</code>	silently accepting a size that does not exist

Run them with `pytest`. If they pass, the numbers in the README are true.

5 The on-ramps — examples/

Three scripts, in increasing order of ambition.

Script	Lines	What it shows
basic_usage.py	77	Start here. Pure Python, no simulator. Single-point solve, catalogue-preset twin, full validation table printed to the terminal.
isaacsim6_actuator.py	156	The reusable class — HarmonicDriveActuator. Drop it into any Isaac Sim robot script: it reads joint state, computes the η -corrected motor torque, applies it, and logs the loss breakdown.
isaacsim6_pendulum.py	270	The complete demo . A 1-DOF pendulum (2 kg arm, 1 m) driven through a real RT1-H-17-100-CS. Builds the scene programmatically, runs headless or with GUI, writes CSV. The arm settles $45^\circ \rightarrow 0^\circ$ in about 3 s at η 80–85 %.

The pendulum example is the one that makes the model *credible to a roboticist* — it proves the twin runs inside a real physics engine at real-time step rates, not only in a notebook.

6 How a user actually meets the package

There are three depths, and most people stop at the first.

- DEPTH 1 · "just give me a number" → `solve_point(30.0, 2000)`
 one line, no classes ~30 seconds to first result
- DEPTH 2 · "model my specific gearbox" → `twin_for_size(17)`, or build a
 swap in your own parameters HarmonicDriveTwin with your own
 MeshParams / FrictionParams
- DEPTH 3 · "put it in my robot sim" → copy `isaacsim6_actuator.py`,
 live actuator in Isaac Sim call `.step()` each frame

7 Before you release — three things to look at

Open decisions, not blockers

- Schaeffler logo files are tracked in a public MIT repository.** `assets/schaeffler_wordmark_green.png` and `we_pioneer_motion_claim.png` are corporate trademarks sitting under an MIT license header. Consider shipping an unbranded PDF publicly and keeping the branded build internal. The same applies to the *INTERNAL — CONFIDENTIAL* footer on these pages.
- LaTeX build artefacts were tracked.** The `.aux`, `.fls`, `.fdb_latexmk`, `.log`, `.out`, `.toc`, `.xdv` and `build.log` files added churn to every commit. Ship `.pdf` and `.tex`; ignore the rest. (*Now fixed.*)
- Uncommitted documentation changes.** Commit or revert everything under `docs/user_guide/` before tagging a release.

Good news: build/, *.egg-info/, .pytest_cache/ and __pycache__/ are already correctly ignored, and the package tree itself is clean.

8 One-page summary

Piece	Size	Role	User opens it?
pyproject.toml	37	shipping label	no
README.md	249	shop window	yes, first
__init__.py	129	front door, public API	indirectly
twin.py	787	the engine	if curious
params.py	425	the dials	yes, to adapt
catalogue.py	250	reality check	to validate
flex_common/ × 7	≈1 300	physics foundation	rarely
examples/ × 3	503	on-ramps	yes, to start
tests/	183	proof	to verify
docs/user_guide.pdf	10 pp	the manual	yes

Dependencies: **NumPy and SciPy only**. Matplotlib optional, for plots. Python ≥ 3.10 . No compiled extensions, no companion repository, no license server. That is why it installs anywhere in one command.